AFSOIS

Agentic Fuel Station Operations & Intelligence System

System Design, Agent Specification & GitHub Blueprint — Version 2.0

Portfolio design and early reference prototype | Synthetic data only

1. System Design

System Design

Executive summary

AFSOIS is a proposed operations-intelligence platform for a retail fuel station. It reconciles physical fuel movement with financial movement using validated records, deterministic calculations, bounded agents, advisory analytics and human oversight. It is a portfolio design and early reference prototype, not a deployed or certified control system.

Goals

• Capture meter readings, dip readings, deliveries, sales, cash/credit allocations and shift handovers.

• Reconcile expected versus measured wetstock and sales versus payment allocations.

• Build evidence-backed incidents and delivery disputes.

• Apply versioned procurement and credit policies.

• Demonstrate responsible agent orchestration using synthetic data.

Non-goals

No live dispenser/PLC/tank-probe/CCTV/fire-system/emergency-shutdown control; no bank transfers, vendor payments, refunds, autonomous credit issuance or ledger edits; no regulatory certification; no production POS, bank or supplier integrations.

Hybrid architecture

```
flowchart TB
 U[Owner / Manager / Attendant] --> UI[Role-based UI]
 UI --> API[FastAPI API + Auth + RBAC]
 API --> WF[Workflow Orchestrator]
 WF --> ORCH[Operations Orchestrator]
 ORCH --> AG[Bounded Specialist Agents]
 AG --> DS[Typed Deterministic Services]
 DS --> DB[(PostgreSQL)]
 WF <--> AP[Human Approval Queue]
 WF --> AU[(Append-only Audit Events)]
 AG --> EV[Evaluation / Tracing]
```

Source-of-truth hierarchy

1. Validated records and evidence with provenance.

2. Deterministic domain calculations.

3. Versioned policy/configuration.

4. Agent interpretation linked to tool outputs.

5. Human-approved correction/disposition.

Key formulas

• `meter_sales = closing_meter - opening_meter`

• `expected_closing = opening + receipts - sales + approved_adjustments`

• `residual = measured_closing - expected_closing`

• `delivery_variance = measured_tank_increase - supplier_claimed_litres`

• `sales_value = validated_litres × effective_unit_price`

Positive residual means measured stock exceeds expected stock. A lower meter closing value must trigger reset/rollover review, not an assumed negative sale. Dip conversion requires a tank-specific versioned chart; no extrapolation. No universal allowable-loss percentage is hardcoded.

Numeric and event integrity

Use Decimal for measured volumes and currency. Store units and currency explicitly. Each event has an event ID and idempotency key. Corrections append reversal/correction events; never overwrite verified history. Evidence files have content hashes and access controls.

Roles

| Role | Access |

|---|---|

| Owner | All records, policy proposals, high-impact approvals |

| Manager | Operational records, shift/delivery/exception review, delegated approvals |

| Attendant | Assigned readings, handovers and checklists |

| Accountant | Financial review and delegated finance approvals only |

| Technician | Equipment/maintenance records |

| Auditor | Time-bounded read-only access |

Enforce permissions in API/data-access layers, not prompts.

Deployment approach

Start as a modular application, PostgreSQL, background worker and isolated agent runtime. Docker Compose is sufficient for a portfolio prototype. Defer Kubernetes until justified.

2. Agent Architecture

Agent Architecture and Orchestration

Topology

```
flowchart TD
 Q[User request / domain event] --> G[Identity, scope and intent validation]
 G --> O[Workflow Orchestrator]
 O --> P{Workflow}
 P -->|Question| A[Read-only Operations Assistant]
 P -->|Stock| W[Wetstock Investigator]
 P -->|Delivery| D[Delivery Evidence Analyst]
 P -->|Cash / credit| R[Reconciliation Analyst]
 P -->|Restock| C[Procurement Advisor]
 P -->|Summary| S[Reporting Agent]
 A --> T[Typed tools / deterministic services]
 W --> T
 D --> T
 R --> T
 C --> T
 S --> T
 T --> V[Schema + policy + evidence validation]
 V --> H{Approval required?}
 H -->|Yes| U[Human review]
 H -->|No| F[Read-only result]
 U --> F
 F --> L[Audit event and trace]
```

Orchestrator duties

Authenticate scope; select allowlisted workflow; load minimum authorised context; call typed tools; enforce timeout/retry/token/delegation limits; validate schema and evidence; route consequential proposals to approval; record trace, policy version and disposition. It cannot bypass domain services or permissions.

Agent contract

```
{
  "schema_version":"1.0",
  "run_id":"run_demo_001",
  "task_id":"task_demo_004",
  "station_scope":"station_demo_01",
  "task_type":"wetstock.investigate",
  "input_refs":["event_demo_1","reading_demo_2"],
  "authority":"read_and_recommend",
  "deadline_seconds":30
}
```

Pass record references, not unrestricted datasets.

State model

`RECEIVED → AUTHENTICATED → CLASSIFIED → DATA_VALIDATED → TOOL_EXECUTION → OUTPUT_VALIDATION → HUMAN_REVIEW (if required) → COMPLETED`

Other terminal states: `REJECTED`, `NEEDS_MORE_INFORMATION`, `FAILED`, `CANCELLED`, `EXPIRED`.

Tool policy

Allow narrow typed tools such as `get_shift_summary`, `calculate_meter_sales`, `calculate_wetstock`, `get_delivery_evidence`, `reconcile_payment_allocations`, `get_policy`, `create_incident_draft`, `request_human_approval`. No unrestricted SQL, shell, arbitrary URL fetch or generic write tool.

Memory

No unconstrained persistent agent memory in MVP. Persist workflow state and record references. Conversation context is scoped by user, station, role and retention policy. Never infer a missing operational fact from chat when the source record is absent.

Failure handling

Retry idempotent reads only; schema invalidity gets at most one constrained repair; missing evidence produces abstention; conflicting records create review cases; retrieved documents are untrusted data; policy service failure blocks consequential actions.

3. Agent Catalogue

Agent Catalogue

Authority levels

• **A0 Observe:** read authorised records.

• **A1 Analyse:** call deterministic read/compute tools and explain.

• **A2 Recommend:** draft proposal; cannot execute.

• **A3 Execute after approval:** a workflow executes the exact approved payload; agent cannot approve itself.

• **Prohibited:** safety hardware control, money movement, silent ledger edits.

1. Operations Orchestrator

Purpose: coordinate requests and events. Inputs: intent, identity/role scope, event refs, policy version. Tools: workflow registry, typed tool router, approval and audit services. Output: workflow state and result refs. Authority: routing only; cannot bypass rules. Escalate ambiguous intent, missing data, policy conflict or failed validation. Evaluate routing, delegation bounds and audit completeness.

2. Operations Assistant — A0/A1

Purpose: answer questions from authorised records. Tools: read-only search, shift/stock summary, policy lookup. Output: answer, source record IDs, missing-data notice and uncertainty. Must not guess deposits, assignments, profit, dues or stock. Evaluate groundedness, abstention and access compliance.

3. Wetstock Investigator — A1/A2

Purpose: investigate expected-versus-measured stock differences. Inputs: opening stock, receipts, meter sales, approved adjustments, closing dip, chart version and sensor provenance. Tools: dip conversion, meter service, wetstock calculation, evidence lookup, incident draft. Output: expected stock, measured stock, signed residual, data-quality status, evidence-backed hypotheses. Must not declare theft/leak/misconduct from variance alone, invent allowable loss or modify stock. Escalate missing/stale dip, chart mismatch, sensor conflict or unexplained residual. Evaluate arithmetic, sign convention, traceability and uncertainty.

4. Delivery Evidence Analyst — A1/A2

Purpose: assemble delivery evidence and identify incomplete/disputed deliveries. Inputs: supplier claim, pre/post dip, chart version, POD, timestamps and recorder. Tools: dip conversion, variance calculator, evidence verifier, incident draft. Output: completeness, measured increase, variance and review status. Must not accept delivery or release payment autonomously. Escalate missing POD, out-of-range dip, timestamp conflict or tolerance breach. Evaluate evidence completeness and false acceptance rate.

5. Reconciliation Analyst — A1/A2

Purpose: compare validated sales with cash, digital payment and credit allocations. Tools: sales-value, payment reconciliation and credit exposure services. Output: expected/recorded totals, variance by channel, missing references and review draft. Must not transfer funds, approve refund, issue credit, write off or edit ledger. Evaluate arithmetic, duplicate detection and unsupported conclusions.

6. Procurement Advisor — A1/A2

Purpose: recommend restocking options. Inputs: stock, demand, lead time, supplier terms, dues and approved policy. Tools: inventory summary, forecast, supplier policy and purchase draft. Output: suggested quantity, assumptions, options, constraint flags and approval draft. Must not place order or override credit policy. Escalate conflicting dues, stale prices, missing inputs or ambiguous rules. Evaluate constraint adherence and freshness.

7. Reporting Agent — A0/A1

Purpose: summarise validated daily/weekly aggregates. Output includes period, completeness, KPI definitions and exception links. Must not call gross margin net profit or invent missing values. Evaluate numerical consistency and disclosure.

Deliberately deterministic capabilities

Meter/sales calculation, dip/inventory calculation, cash/credit ledger, policy/approval, safety notification/runbook and workforce constraint solver are services/workflows, not LLM agents.

4. Shift Close Workflow

Shift Close and Reconciliation

```
sequenceDiagram
 actor Attendant
 participant API
 participant Meter as Meter Service
 participant Finance as Reconciliation Service
 participant Agent as Analyst
 participant Manager
 participant Audit
 Attendant->>API: Submit closing readings and payment counts
 API->>API: Authenticate, validate scope and idempotency
 API->>Meter: Calculate meter delta and sales
 Meter-->>API: Validated litres and value
 API->>Finance: Reconcile allocations
 Finance-->>API: Matched totals and variance
 API->>Agent: Analyse validated result references
 Agent-->>API: Explanation and review recommendation
 API->>Manager: Review task when required
 Manager->>API: Approve / reject / request evidence
 API->>Audit: Append submission and decision events
 API-->>Attendant: Status and exceptions
```

Machine-level meter accounting remains distinct from shift/operator accountability. Shared-machine staffing must be represented explicitly.

5. Wetstock and Delivery Workflow

Wetstock and Delivery

```
flowchart TD
 A[Opening stock and receipts] --> B[Validated meter sales]
 B --> C[Measured closing dip]
 C --> D{Valid readings and chart version?}
 D -->|No| E[INCOMPLETE / REVIEW_REQUIRED]
 D -->|Yes| F[Deterministic expected-stock calculation]
 F --> G[Signed residual]
 G --> H[Wetstock Investigator]
 H --> I{Unexplained variance?}
 I -->|No| J[Routine close]
 I -->|Yes| K[Evidence-backed incident draft]
 K --> L[Human review]
```

Delivery sequence: create draft; capture pre-dip and chart version; record offload/POD; capture post-dip; calculate tank increase; compare with approved versioned tolerance; mark review/dispute if evidence incomplete or variance exceeds policy; human disposition. No payment release is automated.

A stock discrepancy is a signal, not proof of theft or leakage. Possible hazardous conditions are escalated to qualified staff under the approved site procedure; AI does not control safety equipment.

6. Procurement and Approval Workflow

Procurement and Human Approval

Procurement: low-stock/request trigger → read stock and recent sales → validate freshness → estimate requirement using approved rule/forecast → inspect supplier terms and dues → apply versioned policy → produce options and assumptions → human review → manual purchasing process.

An approval request binds action type, exact payload hash, requester, rationale, evidence refs, policy version, risk tier and expiry. Server revalidates role, policy, hash and expiry before execution. Any payload change invalidates approval. The agent that drafts a proposal cannot approve it.

7. Conceptual Data Model

Conceptual Data Model

```
erDiagram
 STATION ||--o{ TANK : has
 STATION ||--o{ MACHINE : has
 TANK ||--o{ DIP_CHART_VERSION : calibrated_by
 TANK ||--o{ DIP_READING : measured
 MACHINE ||--o{ NOZZLE : contains
 NOZZLE ||--o{ METER_READING : reports
 SHIFT ||--o{ SHIFT_ASSIGNMENT : assigns
 SUPPLIER ||--o{ DELIVERY : supplies
 DELIVERY ||--o{ EVIDENCE : supported_by
 CUSTOMER ||--o{ CREDIT_TRANSACTION : has
 SHIFT ||--o{ PAYMENT_ALLOCATION : allocates
 WORKFLOW ||--o{ APPROVAL : requires
 WORKFLOW ||--o{ AUDIT_EVENT : records
```

Core entities: stations, users/roles, tanks, dip-chart versions/points, dip readings, machines/nozzles, meter readings, shifts/assignments/handovers, sales/payment allocations, customers/credit transactions, suppliers/purchase requests/orders/deliveries, evidence, cash counts/deposits/reconciliations, incidents, approvals, agent runs, tool invocations and audit events.

Every station record carries station scope. Measurements carry unit, timestamp, source/recorder and validation state. Dip readings reference chart version. Financial amounts carry currency/decimal precision. Agent outputs reference source IDs and never become source records.

8. API Contract Outline

API Contract Outline

All endpoints require authentication, server-side role/station scope and versioned routes. Event submissions use idempotency keys.

| Method | Endpoint | Purpose |

|---|---|---|

| POST | `/api/v1/shifts/{id}/meter-readings` | Submit meter reading |

| POST | `/api/v1/tanks/{id}/dip-readings` | Submit dip with chart version |

| POST | `/api/v1/deliveries` | Create delivery draft |

| POST | `/api/v1/deliveries/{id}/evidence` | Attach POD/evidence |

| POST | `/api/v1/shifts/{id}/close` | Submit shift close |

| GET | `/api/v1/shifts/{id}/reconciliation` | Read reconciliation |

| GET | `/api/v1/tanks/{id}/wetstock` | Read wetstock result |

| POST | `/api/v1/approvals` | Request approval |

| POST | `/api/v1/approvals/{id}/decision` | Human approval decision |

| POST | `/api/v1/assistant/query` | Read-only assistant |

Example response: `{"status":"REVIEW_REQUIRED","expected_closing_litres":"5500.000","measured_closing_litres":"5480.000","residual_litres":"-20.000","evidence_refs":["dip_demo_22","meter_demo_81"],"policy_version":"demo-2026-01"}`.

Errors: 400 schema/unit; 401 unauthenticated; 403 unauthorised; 404 out of scope/not found; 409 idempotency/stale approval; 422 domain validation; 503 policy/calculation dependency unavailable. This is an API design, not a running API in this release.

9. Security and Threat Model

Security and Threat Model

Assets and trust boundaries

Assets: operational readings, sales, cash/credit, supplier/customer records, employee assignments, evidence files, credentials, policy and audit history. Boundaries: client/API; API/database/object store; orchestrator/agent runtime; agent/tools; external documents/retrieval; approval UI/action executor.

| Threat | Control |

|---|---|

| Prompt injection in supplier PDF | Retrieved content is data, never executable instruction; tool allowlist |

| Excessive agency | No payment, hardware, unrestricted SQL or arbitrary write tools |

| Cross-role data leak | Server-side RBAC and station/row scope |

| Hallucinated deposit/stock fact | Evidence-backed answers; abstain if source absent |

| Duplicate event/replay | Idempotency keys and unique constraints |

| Tampered evidence | Content hash, metadata and restricted object access |

| Approval substitution | Payload hash binding, expiry and revalidation |

| Insider changes old records | Append-only corrections and audit trail |

| Policy/model outage | Fail closed for consequential actions |

| Data exfiltration | Scoped retrieval, redaction and export permissions |

Baseline: TLS; encryption at rest; short-lived tokens; least-privilege service identities; secrets outside Git; structured logs with redaction; rate limits; upload scanning; dependency checks; backups and restore tests. These are deployment requirements, not claims that the portfolio prototype already implements them.

No agent can operate dispensers, fire systems, emergency shutdowns, bank transfers or unrestricted SQL.

10. Safety and Human Oversight

Safety and Human Oversight

1. AI is advisory; physical safety systems remain independent.

2. Never let an LLM directly operate dispensers, pumps, valves, electrical controls, alarms or emergency shutdown equipment.

3. A variance is not a diagnosis. Do not infer theft, leakage, fraud or misconduct without evidence and human investigation.

4. For a reported hazard, the system may notify and log; a qualified person follows the approved site procedure.

5. Consequential actions require server-side role checks and human approval.

| Risk tier | Example | Handling |

|---|---|---|

| Low | Read-only daily summary | Automated response with source refs |

| Medium | Variance explanation / forecast | Advisory; uncertainty shown |

| High | Delivery dispute / credit exception / purchase proposal | Human review |

| Critical | Possible leak, fire or emergency | Qualified human and independent safety system; AI notify-and-record only |

If identity, policy, evidence or calculation cannot be validated, block the action. Separate facts, calculations, hypotheses and recommendations in the UI.

11. Privacy and Data Policy

Privacy and Data Policy

Public portfolio rule

All examples are fabricated. Do not commit real station names/addresses, staff or customer names, phone numbers, supplier identities, bank details, balances, transactions, tank charts, photos, CCTV, credentials or raw operational logs.

Minimise data, use opaque synthetic IDs and redact personal/financial fields before model calls. A real deployment would require a documented lawful basis, retention schedule, access/deletion process, incident response and local legal review. Review model-provider retention, training-use, region, access and contractual terms before sending any real data.

Publication checklist

• [ ] Search names, emails, phone numbers and addresses.

• [ ] Search bank/account identifiers and credentials.

• [ ] Confirm station/event IDs are synthetic.

• [ ] Confirm sample dip charts and values are fabricated.

• [ ] Remove raw source documents and screenshots.

• [ ] Review Git history for secrets before public release.

12. Evaluation Plan

Agent and System Evaluation Plan

Layers

• Deterministic services: unit tests, property tests, arithmetic regression and boundary cases.

• Agents: tool selection, schema validity, evidence grounding, abstention, prompt-injection resistance, authority compliance and escalation.

• Workflows: idempotency, retries, timeouts, approval payload integrity and partial-failure recovery.

• Future ML: temporal splits, leakage checks, baseline comparison, calibration and drift.

Proposed portfolio acceptance targets (not measured claims)

| Metric | Proposed target |

|---|---|

| Deterministic arithmetic suite | 100% pass |

| Defined invalid-input cases | 100% rejected |

| Agent schema validity | ≥99% on fixed eval set |

| Unsupported critical claims | 0 on golden tests |

| Prohibited actions | 100% blocked by server/tool boundary |

| Missing-data abstention | ≥95% on eval set |

| Approval audit trace completeness | 100% on workflow tests |

Golden scenarios

1. Missing closing dip → incomplete, not estimated.

2. Chart version mismatch → no conversion.

3. Closing meter below opening → reset/rollover review.

4. Duplicate delivery event → no duplicate receipt.

5. Cash mismatch → show category difference; no transfer.

6. Attendant requests owner-only credit data → deny or authorised aggregate.

7. Supplier PDF prompt injection → ignore embedded instructions.

8. Conflicting vendor dues → ask for reconciliation; no purchase.

9. Possible leak → notify qualified person; no diagnosis/control.

10. Approval payload changed → invalidate approval.

11. Historical correction → append reversal, preserve original.

12. Gross margin labelled net profit → evaluation failure.

Do not promote an agent beyond read/recommend until its eval suite, audit behaviour and permission boundaries pass.

13. Product Requirements

Product Requirements Brief

Problem

Fuel-station workflows combine physical measurement, machine readings, shifts, deliveries, payments, credit and supplier obligations. Records may be fragmented. Operators need timely reconciliation and explainable exceptions.

Users

Owner (policy, approvals, reporting); manager (shift close and exceptions); attendant (assigned readings and handover); accountant (financial review); technician (maintenance); auditor (read-only traceability).

MVP stories

• Attendant submits opening/closing meter and dip readings for assigned shift.

• Manager sees shift completeness and missing records.

• Owner inspects expected versus measured stock with formula and evidence.

• Accountant compares validated sales with payment allocations.

• Manager reviews delivery evidence and opens a dispute.

• Owner asks questions and receives answers linked to authorised records.

• Approver accepts/rejects a specific proposal with audit record.

Principles

Accuracy before automation; explain every discrepancy; human authority for consequential actions; least privilege; synthetic data in public demo; autonomy only after evaluation.

Future success measures

Shift completeness; time from close to reconciliation; evidence completeness; duplicate-event rate; anomaly false-positive/negative rates; human override/rejection; user trust. No impact claim until measured in a controlled pilot.

14. Implementation Roadmap

Implementation Roadmap

Phase 0 — Design and reference prototype (current)

Architecture, agent specs, workflows, governance, synthetic fixtures and small tested Python calculation services.

Phase 1 — Deterministic operations core

Schemas, versioned dip-chart registry, meter/sales, wetstock, payment allocation, idempotent ingestion, PostgreSQL/migrations, synthetic simulator. **Exit:** core truth and arithmetic work without LLM.

Phase 2 — Workflow and approval

Shift close, delivery verification, payload-hash approvals, append-only audit, RBAC, retries and recovery. **Exit:** full audit trace.

Phase 3 — Read-only agents

Operations Assistant, Wetstock Investigator, Delivery Evidence Analyst, Reconciliation Analyst; retrieval with citations and golden evaluation set. **Exit:** grounding, abstention, schema and permission gates pass.

Phase 4 — Recommendations and advisory ML

Procurement Advisor, Reporting Agent, demand forecast and anomaly baselines, monitoring and drift. **Exit:** assumptions visible; human review required.

Phase 5 — UI and pilot preparation

Role-based dashboard, exception inbox, approval screen, synthetic demo, usability/security review and qualified local legal/metrology/fire-safety/privacy review before any real deployment.

No live hardware or financial integrations are included in this portfolio scope.

15. Decision Register

Decision Register and Source Ambiguities

| ID | Topic | Source issue | Design treatment |

|---|---|---|---|

| D-01 | Tank capacity | Source notes contain conflicting petrol/diesel capacity figures and chart headings | Do not treat as verified; synthetic chart fixtures only |

| D-02 | Dip chart | Geometry/calibration provenance incomplete | Tank-specific versioned chart; no extrapolation |

| D-03 | Allowable loss | Suggested range not established as universal | Unconfigured until qualified local validation and approval |

| D-04 | Wetstock equation | Differing sign conventions and inconsistent worked arithmetic | Use documented equation and signed residual; regression tests |

| D-05 | Vendor dues | Aggregate versus per-vendor threshold unclear | Business owner must define and version policy |

| D-06 | Customer credit | Example balances do not reconcile to aggregate | Recalculate from transaction ledger; synthetic only |

| D-07 | Profit terminology | Per-litre margin confused with total margin | Separate per-litre gross margin and total gross margin; net profit needs other costs |

| D-08 | Machine/operator | Shared-machine attribution needs separate dimensions | Keep machine meter and operator/shift accountability distinct |

| D-09 | Safety response | Earlier notes suggested AI-triggered till lock/safety actions | Excluded; notify and record only |

| D-10 | Sensor priority | Probe/dip priority varies across notes | Store provenance; conflict requires review |

Before a real pilot, confirm tank IDs/capacity/geometry/calibration; approve chart versions; define tolerance/loss policy; resolve vendor dues and customer credit policies; define meter reset/rollover; define business-day timezone; review retention and privacy.

16. LinkedIn and Resume Copy

LinkedIn and Resume Copy

LinkedIn project entry

AFSOIS — Agentic Fuel Station Operations & Intelligence System | Independent System Design Project

Designed a hybrid agentic-AI architecture for fuel-station operations covering wetstock monitoring, meter and sales reconciliation, delivery verification, cash/credit review and procurement decision support. Defined a workflow orchestrator with bounded specialist agents, deterministic calculation services, typed tool contracts, role-based permissions, human approval gates and auditability. Produced architecture, agent specifications, workflow diagrams, conceptual data model, API contracts, threat model, evaluation plan and synthetic reference datasets. Built small Python reference functions for meter and wetstock calculations with unit tests.

Technology/design: Python, Decimal arithmetic, pytest, Mermaid, GitHub Actions; FastAPI, PostgreSQL and LangGraph/state-machine orchestration in the proposed architecture.

Status: System design and early reference-service prototype; full platform and agent runtime not yet implemented.

Resume bullets

• Designed AFSOIS, a hybrid agentic-AI system for fuel-station operations, combining deterministic inventory/financial calculations, bounded specialist agents and human-in-the-loop approvals.

• Authored agent specifications, workflows, data model, API contracts, security/threat model and evaluation strategy for wetstock, delivery, shift-close and reconciliation.

• Built tested Python reference services for meter deltas, sales values, dip-chart interpolation and wetstock reconciliation using Decimal arithmetic and synthetic data.

Product / AI systems emphasis

• Translated operational workflows into a system design defining product scope, roles, exception handling, agent boundaries, approval flows and phased roadmap.

• Separated deterministic operational truth from LLM-based evidence synthesis and recommendations.

• Defined least-privilege tools, server-side RBAC, grounded outputs, approval payload integrity, audit events and exclusion of AI control over safety-critical hardware or financial transfers.

LinkedIn post

I’ve been working on AFSOIS, an independent system-design project exploring how agentic AI can support fuel-station operations.

The challenge is to bring together physical inventory, meter readings, deliveries, shift records and financial reconciliation without allowing an AI model to become the source of truth.

I designed a hybrid architecture with deterministic calculation services, a workflow orchestrator, bounded specialist agents and human approval for consequential decisions. The project includes agent specifications, workflows, data model, API outline, governance controls, evaluation plan and synthetic examples, alongside a small tested Python reference layer.

The key principle: **AI should help investigate and explain operational exceptions; verified services and accountable people should remain in control.**

This is a system-design portfolio project and early prototype, not a production deployment. Feedback on the architecture and agent boundaries is welcome.

Interview explanation

"I started with the operational workflow rather than adding agents everywhere. I separated physical-stock calculations and financial reconciliation into deterministic services. Agents are used where language understanding and evidence synthesis add value, such as investigating stock variance or explaining delivery discrepancies. I added strict permissions, approval gates and auditability because an operational AI system must not independently move money, alter records or control safety equipment."

17. Publication Checklist

Publication Checklist

• [ ] Use synthetic examples only; remove source docs and raw operational records.

• [ ] Search names, phone numbers, emails, addresses, bank identifiers and secrets.

• [ ] Confirm all event/station IDs and chart values are fabricated.

• [ ] Review Git history for accidental secrets before making public.

• [ ] Run tests and record actual result.

• [ ] Verify README status claims and licence ownership.

• [ ] Add description and topics.

Suggested repository name: `afsois-agentic-fuel-station-operations`

Description: `Portfolio system design for a hybrid agentic-AI fuel-station operations platform. Synthetic data only.`

Topics: `agentic-ai`, `system-design`, `python`, `operations`, `workflow-orchestration`, `human-in-the-loop`, `synthetic-data`

Suggested tag: `v0.2.0-design-and-reference-prototype`